PORTFOLIO CASE STUDY · SQL SERVER · CUSTOMER ANALYTICS

RFM Customer Segmentation with SQL Server

An end-to-end customer analytics project using AdventureWorks2022 to transform transactional sales data into actionable customer segments.

CUSTOMERS ANALYZED
19,119

SCORING METHOD
NTILE(5)

PRIMARY TECHNIQUES
CTEs · Joins · Window Functions

SEGMENTS CREATED
4 actionable groups

Aref Navabi
Data & AI Portfolio Project

01 · PROJECT OVERVIEW

From transactional data to customer strategy

RFM analysis is a practical framework for ranking customers by **Recency**, **Frequency**, and **Monetary value**. This project applies the framework directly in SQL Server, using AdventureWorks2022 sales transactions to build customer-level features, assign RFM scores, create business-oriented segments, and summarize their commercial significance.

19,119
CUSTOMERS IN FINAL ANALYSIS

73.82%
MONETARY VALUE FROM HIGH VALUE

93.18%
HIGH VALUE + AT RISK COMBINED

Business question

Which customers are currently most valuable, which valuable customers may be drifting away, and where should marketing effort be prioritized?

Analytical objective

Create a reproducible SQL pipeline that calculates RFM metrics, scores customers from 1 to 5, assigns interpretable segments, and produces segment-level KPIs.

Project scope

SQL Server · AdventureWorks2022 · Customer Segmentation · RFM · Window Functions · Business Insights

The analysis intentionally stays inside SQL Server. The goal is not to build a machine-learning clustering model, but to demonstrate a transparent segmentation workflow that can be understood, audited, and adapted by analysts and business stakeholders.

> **Reference date:** rather than using the current date against historical data, the analysis uses one day after the latest order date. The maximum order date is 2014-06-30, so the effective reference date is 2014-07-01.

Building the RFM feature set

Sales.SalesOrderHeader

Key fields: SalesOrderID, OrderDate, CustomerID, TotalDue

Used for: customer identification, last purchase date, and number of unique orders.

Sales.SalesOrderDetail

Key fields: SalesOrderID, ProductID, OrderQty, UnitPrice, LineTotal

Used for: monetary value, calculated from the sum of LineTotal.

R
Recency
Days since the customer's most recent order

→

F
Frequency
Count of distinct orders per customer

→

M
Monetary
Sum of line-level purchase value

Metric definitions

Metric	SQL logic	Interpretation
Recency	`DATEDIFF(DAY, MAX(OrderDate), @ReferenceDate)`	Lower is better: a smaller value means a more recent purchase.
Frequency	`COUNT(DISTINCT SalesOrderID)`	Higher is better: more unique orders indicate stronger repeat behavior.
Monetary	`SUM(LineTotal)`	Higher is better: greater historical purchase value.

Why DISTINCT matters: an order can contain multiple line items. Counting rows after joining header and detail tables would overstate purchase frequency. Frequency is therefore calculated from distinct order IDs at the customer level.

Why LineTotal: Monetary is based on product line value, not TotalDue. This keeps the metric aligned with the project definition and excludes components such as freight and tax that may be included in the order total.

03 · SQL IMPLEMENTATION

A structured, reproducible SQL pipeline

The implementation separates the analysis into logical CTEs: one for Recency/Frequency, one for Monetary, one to combine the metrics, and one to apply RFM scoring. The final result is stored in a temporary table so both customer-level and segment-level outputs can be queried without repeating the full pipeline.

1. Reference date + Recency/ Frequency

```
DECLARE @ReferenceDate DATE;
SELECT @ReferenceDate = DATEADD(DAY, 1,
MAX(OrderDate))
FROM Sales.SalesOrderHeader;

;WITH RF AS (
  SELECT CustomerID,
    DATEDIFF(DAY, MAX(OrderDate), @ReferenceDate)
AS Recency,
    COUNT(DISTINCT SalesOrderID) AS Frequency
  FROM Sales.SalesOrderHeader
  GROUP BY CustomerID
)
```

2. Monetary value

```
M AS (
  SELECT h.CustomerID,
         SUM(d.LineTotal) AS Monetary
  FROM Sales.SalesOrderHeader AS h
  INNER JOIN Sales.SalesOrderDetail AS d
    ON h.SalesOrderID = d.SalesOrderID
  GROUP BY h.CustomerID
)
```

3. Window-based scoring

```
NTILE(5) OVER (
  ORDER BY Recency DESC, CustomerID
) AS R_Score,

NTILE(5) OVER (
  ORDER BY Frequency ASC, CustomerID
) AS F_Score,

NTILE(5) OVER (
  ORDER BY Monetary ASC, CustomerID
) AS M_Score
```

4. Business segmentation

```
CASE
  WHEN R_Score <= 2
   AND F_Score >= 4
   AND M_Score >= 4 THEN 'At Risk'
  WHEN RFM_TotalScore >= 12 THEN 'High Value'
  WHEN RFM_TotalScore >= 8  THEN 'Medium Value'
  ELSE 'Low Value'
END AS CustomerSegment
```

Techniques demonstrated

CTEs	INNER JOIN	GROUP BY	Aggregations
DATEDIFF	NTILE	CASE	Temp Tables

04 · SCORING & SEGMENTATION DESIGN

Turning raw metrics into interpretable customer groups

Why NTILE(5)?

NTILE(5) provides a direct 1-5 score for each RFM dimension and is easy to communicate. It keeps every customer row while assigning a relative position within the customer population.

Direction of scoring

Recency: lower raw values should receive higher scores. **Frequency and Monetary:** higher raw values should receive higher scores.

Segmentation rules

Segment	Rule	Business meaning
At Risk	R ≤ 2 and F ≥ 4 and M ≥ 4	Historically valuable and frequent, but no recent purchase.
High Value	Total RFM score ≥ 12	Strong overall RFM profile and highest priority for retention.
Medium Value	Total RFM score ≥ 8	Moderate value with potential to grow.
Low Value	All remaining customers	Lower engagement and historical purchase value.

Why At Risk is evaluated first: a customer can have strong Frequency and Monetary scores but weak Recency. Without prioritizing the At Risk rule, that customer could be labeled High Value or Medium Value based only on the total score, hiding an important retention signal.

Deterministic tie handling

CustomerID is included as the second sort key in each NTILE expression. This makes repeated executions reproducible when multiple customers have identical Recency, Frequency, or Monetary values.

Limitation: NTILE aims for approximately equal-sized buckets. Equal metric values can still be split across adjacent tiles at a boundary. The tie-breaker makes the result stable, but it does not make NTILE tie-preserving.

05 · RESULTS

Segment performance at a glance

Segment	Customers	Avg Recency	Avg Frequency	Avg Monetary	Monetary Share
High Value	4,006	93.57	3.02	20,242.22	**73.82%**
At Risk	2,003	310.19	2.26	10,615.06	**19.36%**
Medium Value	6,557	164.13	1.26	902.43	5.39%
Low Value	6,553	241.80	1.01	240.63	1.44%

Share of total monetary value

High Value **73.82%**

At Risk **19.36%**

Medium Value **5.39%**

Low Value **1.44%**

Share of customer base

High Value **20.95%**

At Risk **10.48%**

Medium Value **34.30%**

Low Value **34.27%**

93.18%

of total monetary value comes from the **High Value** and **At Risk** segments combined. This concentration makes retention and reactivation the strongest strategic priorities in the analysis.

Total monetary value across all four segments: approximately 109.85M, based on the sum of SalesOrderDetail.LineTotal.

What each segment means for action

High Value — 73.82%

4,006 customers · Avg F 3.02 · Avg M 20,242.22

Interpretation: the strongest overall customer group and the largest source of purchase value.
Action: prioritize retention, loyalty programs, VIP treatment, cross-sell, and upsell.

At Risk — 19.36%

2,003 customers · Avg R 310.19 days · Avg M 10,615.06

Interpretation: valuable historical customers whose recent inactivity may indicate churn risk.
Action: targeted win-back campaigns, personalized offers, and re-engagement outreach.

Medium Value — 5.39%

6,557 customers · Avg F 1.26 · Avg M 902.43

Interpretation: a large pool with moderate value and room to increase repeat behavior.
Action: second-purchase incentives, relevant cross-sell offers, and frequency-building campaigns.

Low Value — 1.44%

6,553 customers · Avg F 1.01 · Avg M 240.63

Interpretation: many customers, but limited historical value and almost no repeat purchasing.
Action: scalable, low-cost automation rather than high-touch marketing investment.

Key business insights

- **Value is highly concentrated:** 20.95% of customers in High Value account for 73.82% of total monetary value.
- **At Risk deserves immediate attention:** only 10.48% of customers represent 19.36% of historical monetary value.
- **Scale does not equal value:** Medium Value and Low Value together represent more than two-thirds of customers, yet contribute less than 7% of monetary value.
- **Marketing should be differentiated:** retention, reactivation, growth, and low-cost nurture strategies should be applied to different segments rather than using one campaign for everyone.

What this project demonstrates

Design choice	**Dynamic reference date:** using one day after the latest order date keeps Recency meaningful for a historical dataset and makes the script reusable if the data changes.
Data quality	**Distinct order counting:** Frequency is calculated before the detail-level join, avoiding inflated counts caused by multiple line items per order.
Reproducibility	**Deterministic NTILE ordering:** CustomerID is used as a secondary sort key so repeated executions produce stable assignments when metric values are tied.
Interpretability	**Rule-based segmentation:** the four segments are transparent and easy to explain to non-technical stakeholders. Thresholds are analytical choices and can be adapted to business objectives.

Limitations

- RFM reflects historical purchasing behavior and does not include profitability, demographics, satisfaction, product preference, or acquisition cost.
- NTILE creates approximately equal-sized groups and can split identical values across adjacent score buckets.
- The segment thresholds are project-specific analytical rules, not universal standards.
- Monetary is based on `LineTotal`, so freight and tax components that may exist in `TotalDue` are not included.

Skills demonstrated

SQL Server	Data Modeling	CTEs	Joins
Aggregations	Window Functions	RFM Analysis	Business Analytics

Conclusion. This project shows how a transparent SQL workflow can turn transaction-level sales data into customer-level behavioral features, reproducible RFM scores, actionable segments, and business recommendations - without leaving the database layer.

Companion code: the GitHub repository is intended to include a clean, commented SQL script containing the complete analysis pipeline.